

Smart Intelligent Cane

System Architecture and Design

Revision A | 20 September 2026 | Status: PROPOSED, NOT VERIFIED

This is the proposed architecture and design for the Smart Intelligent Cane, from the physical level up to the application level. It is a design proposal, not a verified schematic: pin functions, part numbers and I2C addresses were written from memory and must be checked against datasheets before anyone builds from them.

The cane has two processors. An ESP32-S3 is always on and owns every safety function. A Raspberry Pi Zero 2 W is a switchable gateway for camera, voice AI and cloud.

Contents: 1. Engineering diagram set (architecture, hardware block diagram, schematic sheets 3A-3C) 2. Part 1: hardware connections and pin allocations 3. Part 2: software architecture, UML and flows 4. Part 3: power distribution and wiring harness 5. Design decisions and open points.

Conventions: TBD = not defined in the project specification. In the schematic, nets with the same name are connected; pin names are inside each IC body and net names are on the stubs outside; red bold nets are power rails. IC pad numbers are left out on purpose until taken from datasheets. Raspberry Pi pin numbers are physical 40-pin header numbers.

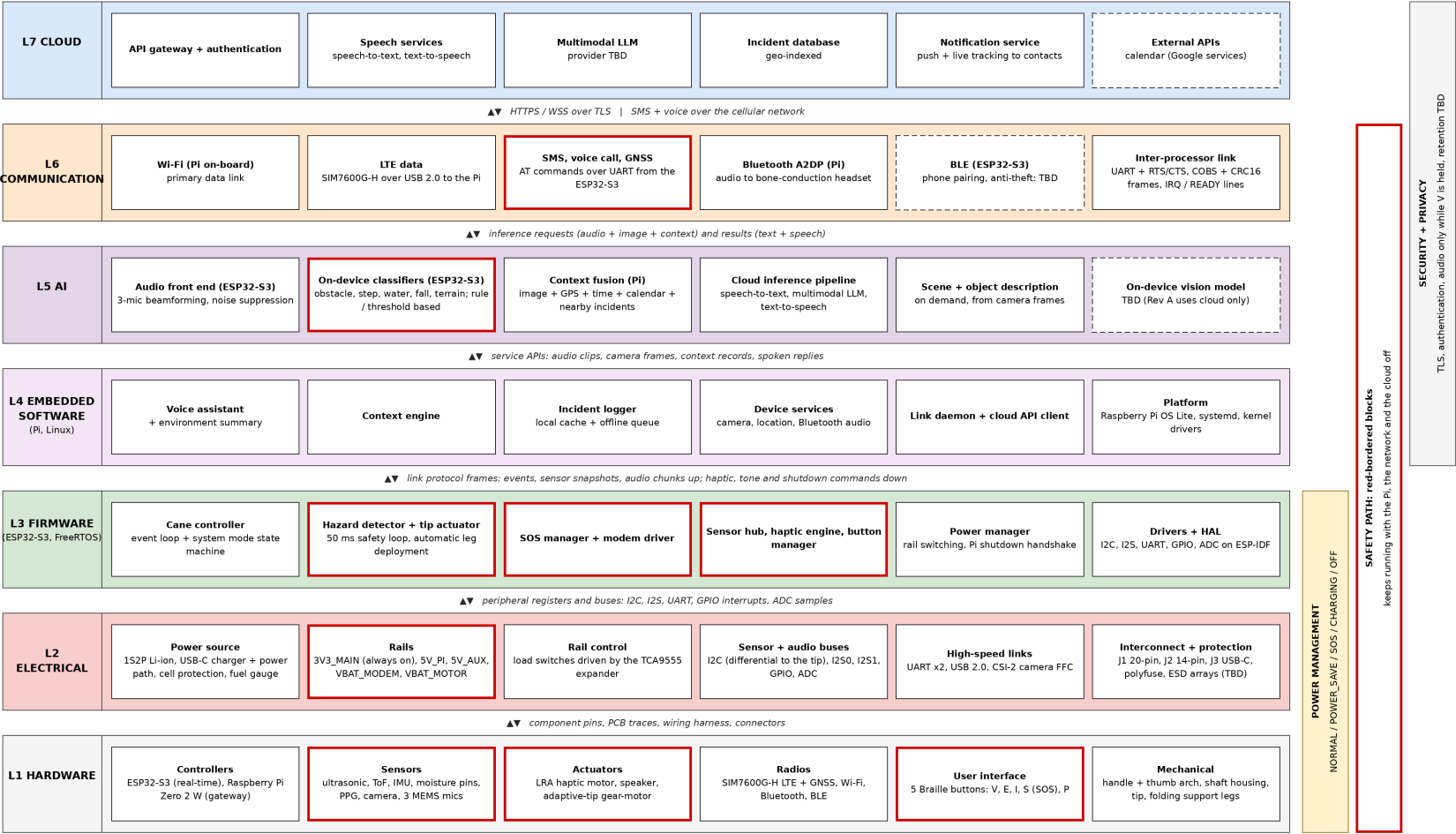
1. Engineering diagram set

1.1 Hierarchical multi-domain architecture

Seven domains stack from hardware up to cloud; the italic strip between two bands names the interface joining them. Red-bordered blocks form the safety path, which keeps running with the Pi, the network and the cloud off.

SMART INTELLIGENT CANE - HIERARCHICAL MULTI-DOMAIN SYSTEM ARCHITECTURE

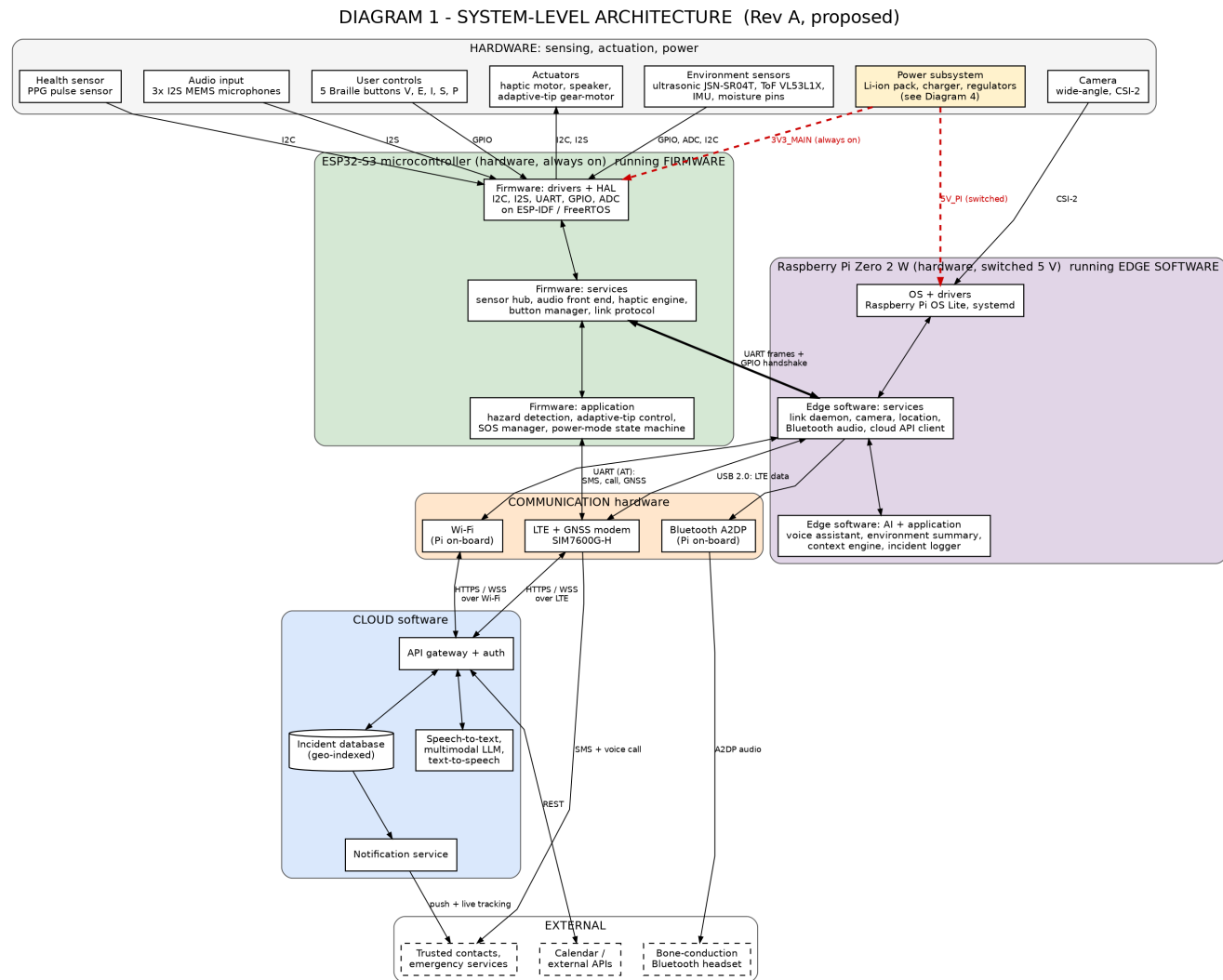
Rev A (proposed). Read bottom-up: each layer uses only the layer below it, through the interface named between them.



LEGEND: band colour = domain | red 3 pt border = safety-critical | dashed border = optional or TBD | italic strip = interface between two layers | vertical bars = cross-cutting concerns and the layers they span

1.2 Diagram 1: system-level architecture

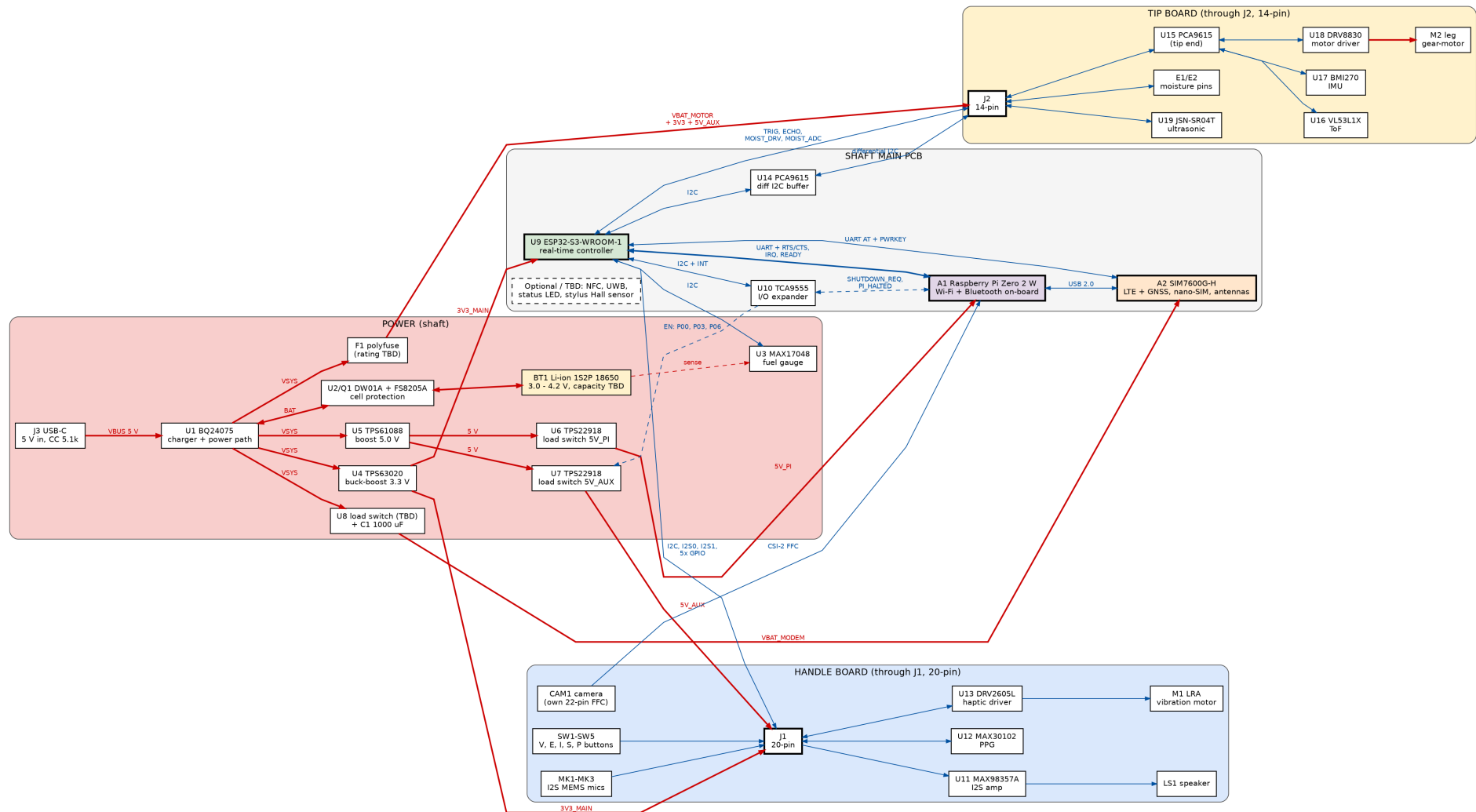
Data flows from sensors through the ESP32-S3 firmware and the Pi edge software to the cloud; each arrow is labelled with its protocol.



1.3 Diagram 2: hardware block diagram

Every physical part appears with its reference designator. Red = power, blue = signal and data, dashed blue = control.

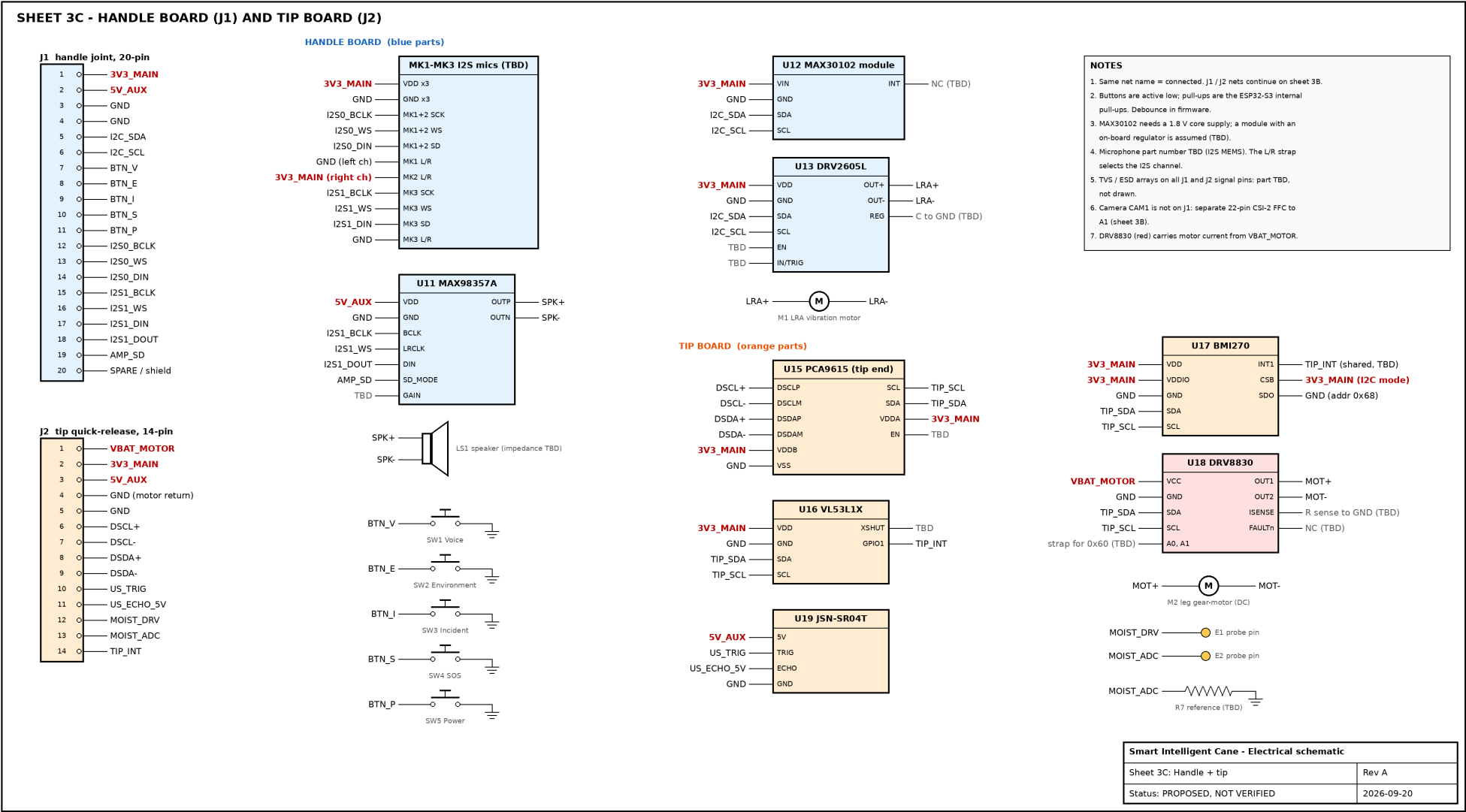
DIAGRAM 2 - HARDWARE BLOCK DIAGRAM red = POWER, blue = SIGNAL / DATA, dashed blue = control (Rev A, proposed)







1.6 Diagram 3, sheet 3C: handle board (J1) and tip board (J2)



2. Part 1: hardware connections

2.1 Hardware interconnect

Every sensor and actuator hangs off the ESP32-S3; the Pi only owns the camera, the modem's data link and the wireless links.

Link	Bus	Between	Notes
Buttons V, E, I, S, P	5 GPIO, active low	Handle to ESP32-S3	SOS and Power can wake the ESP32-S3 from deep sleep
PPG and haptic driver	I2C	Handle to ESP32-S3	MAX30102 and DRV2605L + LRA
Mics 1 + 2	I2S0	Handle to ESP32-S3	Stereo pair on one data line
Mic 3 + speaker amp	I2S1, full duplex	Handle to ESP32-S3	MAX98357A amp shares the clocks
Camera	CSI-2, 22-pin FFC	Handle to Pi	Separate from J1, under 200 mm
Inter-processor link	UART + RTS/CTS, IRQ, READY	ESP32-S3 to Pi	Framed protocol, carries audio chunks too
Modem control	UART, AT commands	ESP32-S3 to SIM7600G-H	SMS, call and GNSS fix with the Pi off
Modem data	USB 2.0	Pi to SIM7600G-H	LTE data for cloud calls
Tip I2C devices	I2C through PCA9615 differential buffer	ESP32-S3 to tip	VL53L1X ToF, BMI270 IMU, DRV8830 motor driver
Ultrasonic	TRIG and ECHO GPIO	ESP32-S3 to tip	JSN-SR04T; ECHO passes a 5 V to 3.3 V divider
Moisture pins	Excite GPIO + ADC	ESP32-S3 to tip	Pulsed excitation to limit electrode corrosion
Headset audio	Bluetooth A2DP	Pi to headset	The ESP32-S3 cannot do this (BLE only)

2.2 Raspberry Pi Zero 2 W: 40-pin header allocation

The Pi uses 10 signal pins: a 4-wire UART to the ESP32-S3, four handshake lines, and 5 V power in. Everything else is free or reserved. All GPIO are 3.3 V logic; never feed 5 V into one.

Pin	Name	Assignment	Group	Pin	Name	Assignment	Group
1	3V3	Not connected	-	2	5V	5V_PI rail in (from load switch)	Power
3	GPIO2 SDA1	Reserved, spare I2C	Reserved	4	5V	5V_PI rail in	Power
5	GPIO3 SCL1	Reserved, spare I2C	Reserved	6	GND	Ground	Power
7	GPIO4	Free	-	8	GPIO14 TXD1	To ESP32 GPIO18 (RX)	UART
9	GND	Ground	Power	10	GPIO15 RXD1	From ESP32 GPIO17 (TX)	UART
11	GPIO17 RTS1	To ESP32 GPIO11 (CTS)	UART	12	GPIO18	Free	-
13	GPIO27	ESP_IRQ in, from ESP32 GPIO12	Handshake	14	GND	Ground	Power
15	GPIO22	PI_READY out, to ESP32 GPIO13	Handshake	16	GPIO23	SHUTDOWN_REQ in, from TCA9555 P01	Handshake
17	3V3	Not connected	-	18	GPIO24	PI_HALTED out, to TCA9555 P02	Handshake
19	GPIO10 MOSI	UWB module (optional)	SPI, optional	20	GND	Ground	Power
21	GPIO9 MISO	UWB module (optional)	SPI, optional	22	GPIO25	UWB IRQ (optional)	SPI, optional
23	GPIO11 SCLK	UWB module (optional)	SPI, optional	24	GPIO8 CE0	UWB chip select (optional)	SPI, optional
25	GND	Ground	Power	26	GPIO7 CE1	Free	-
27	ID_SD	HAT EEPROM, do not use	-	28	ID_SC	HAT EEPROM, do not use	-
29	GPIO5	Free	-	30	GND	Ground	Power
31	GPIO6	Free	-	32	GPIO12	Free	-
33	GPIO13	Free	-	34	GND	Ground	Power
35	GPIO19	Free	-	36	GPIO16 CTS1	From ESP32 GPIO10 (RTS)	UART
37	GPIO26	Free	-	38	GPIO20	Free	-
39	GND	Ground	Power	40	GPIO21	Free	-

Off-header ports: the CSI-2 connector (22-pin FFC) goes to the handle camera, USB OTG data (or test pads, TBD) goes to the SIM7600G-H, and the microSD holds the OS.

```
# /boot/firmware/config.txt
enable_uart=1                # mini-UART on GPIO14/15; Bluetooth stays free for A2DP
dtoverlay=gpio-shutdown,gpio_pin=23 # ESP32 asks the Pi to shut down cleanly
dtoverlay=gpio-poweroff,gpiopin=24  # Pi signals it has halted, so 5V_PI can be cut
```

2.3 ESP32-S3-WROOM-1 (N8R8): pin allocation

The ESP32-S3 uses 28 GPIO directly; slow control signals go through a TCA9555 I2C expander because the module runs out of safe pins.

GPIO	Signal	Direction and destination	Group	GPIO	Signal	Direction and destination	Group
1	MOIST_ADC	In, moisture sense pin (ADC1_CH0)	Tip	15	BTN_S	In, SOS button, deep-sleep wake	Buttons
2	MOIST_DRV	Out, moisture excite pin, pulsed	Tip	16	BTN_P	In, Power button, deep-sleep wake	Buttons
3	EXP_INT	In, TCA9555 interrupt (open drain, pulled up)	Expander	17	U1_TX	Out, to Pi GPIO15 RXD (pin 10)	Pi UART
4	US_TRIG	Out, ultrasonic trigger via J2	Tip	18	U1_RX	In, from Pi GPIO14 TXD (pin 8)	Pi UART
5	US_ECHO	In, ultrasonic echo via 5 V to 3.3 V divider	Tip	21	I2S1_DIN	In, mic 3 data	Audio
6	BTN_V	In, Voice AI button	Buttons	38	I2S1_BCLK	Out, mic 3 + amp	Audio
7	BTN_E	In, Environment button	Buttons	39	I2S1_WS	Out, mic 3 + amp	Audio
8	I2C_SDA	Bidirectional, all I2C devices	I2C	40	I2S0_BCLK	Out, mics 1 + 2	Audio
9	I2C_SCL	Out, all I2C devices	I2C	41	I2S0_WS	Out, mics 1 + 2	Audio
10	U1_RTS	Out, to Pi GPIO16 CTS (pin 36)	Pi UART	42	I2S0_DIN	In, mics 1 + 2 data	Audio
11	U1_CTS	In, from Pi GPIO17 RTS (pin 11)	Pi UART	43	U2_TX	Out, to SIM7600 RXD (AT commands, SOS path)	Modem
12	ESP_IRQ	Out, to Pi GPIO27 (pin 13)	Pi handshake	44	U2_RX	In, from SIM7600 TXD	Modem
13	PI_READY	In, from Pi GPIO22 (pin 15)	Pi handshake	47	I2S1_DOUT	Out, to MAX98357A amp	Audio
14	BTN_I	In, Incident button	Buttons	48	SIM_PWRKEY	Out, SIM7600 power key	Modem

Pins deliberately left alone: GPIO0 (boot button), GPIO19 and 20 (native USB for flashing and debug), GPIO26 to 37 (flash and octal PSRAM on the N8R8 module), GPIO45 and 46 (boot strapping).

2.4 I2C address map (one bus, GPIO8 and 9)

Address	Device	Location
0x20	TCA9555 I/O expander	Shaft
0x24	PN532 NFC (optional)	Shaft
0x29	VL53L1X ToF	Tip
0x36	MAX17048 fuel gauge	Shaft
0x57	MAX30102 PPG	Handle
0x5A	DRV2605L haptic driver	Handle
0x60	DRV8830 leg motor driver	Tip
0x68	BMI270 IMU	Tip

Tip devices sit behind the PCA9615 differential buffer because plain I2C is unreliable over the 1.1 m cable run. Run that segment at 100 kHz.

2.5 TCA9555 port map (slow control signals)

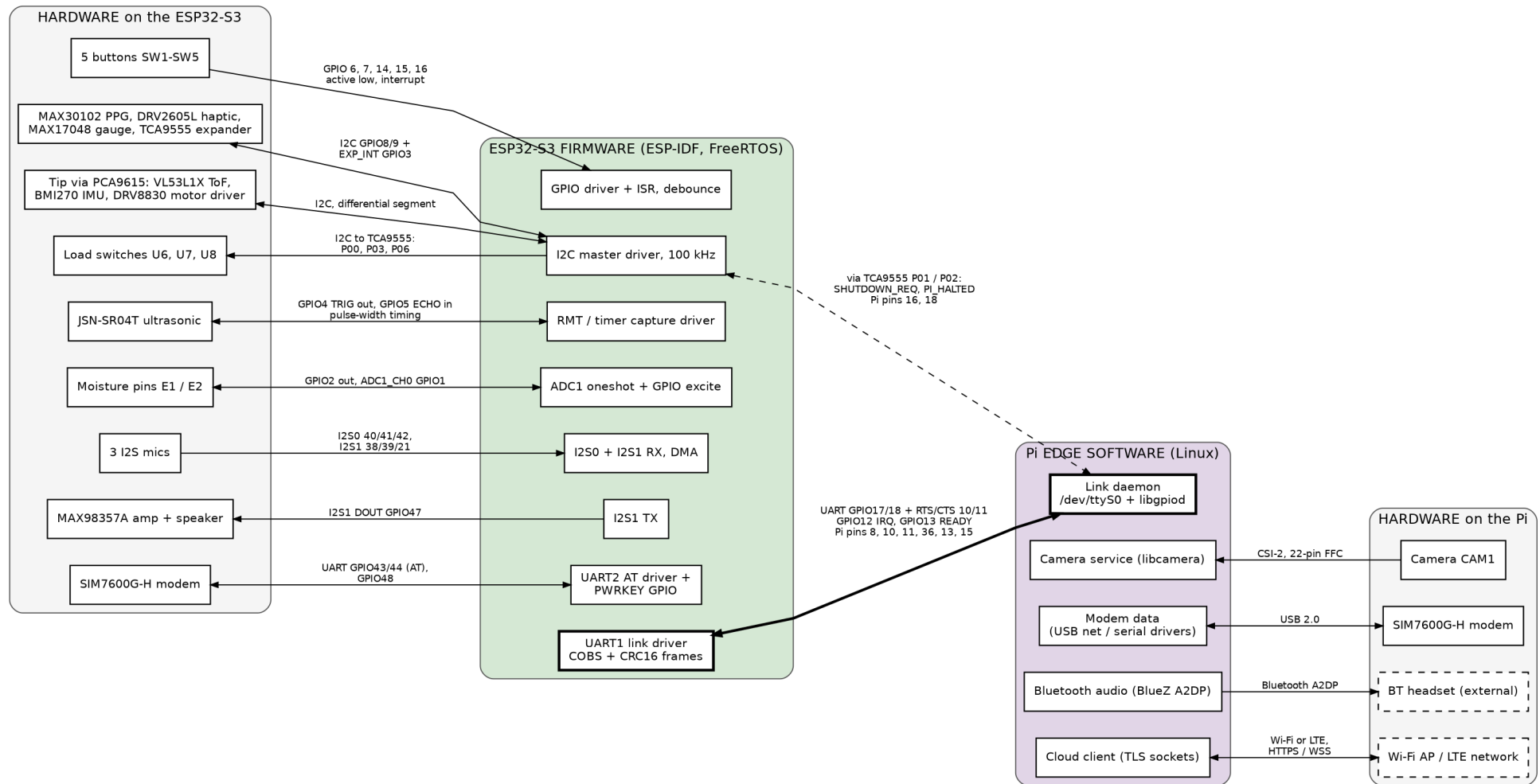
Port	Signal	Direction and destination
P00	PI_PWR_EN	Out, 5V_PI load switch
P01	SHUTDOWN_REQ	Out, Pi GPIO23
P02	PI_HALTED	In, Pi GPIO24
P03	AUX_5V_EN	Out, 5V_AUX load switch
P04	AMP_SD	Out, MAX98357A shutdown
P05	SIM_STATUS	In, SIM7600 status
P06	MODEM_PWR_EN	Out, VBAT modem load switch
P07	CHG_STAT	In, charger status
P10	STYLUS_HALL	In, stylus dock sensor (TBD)
P11	TIP_INT	In, ToF and IMU interrupt via J2
P12 to P17	Spare	-

3. Part 2: software

3.1 Diagram 5: hardware / software interface

Each firmware driver is tied to the hardware it controls, and each Pi service to its device, with the protocol and pins on every link.

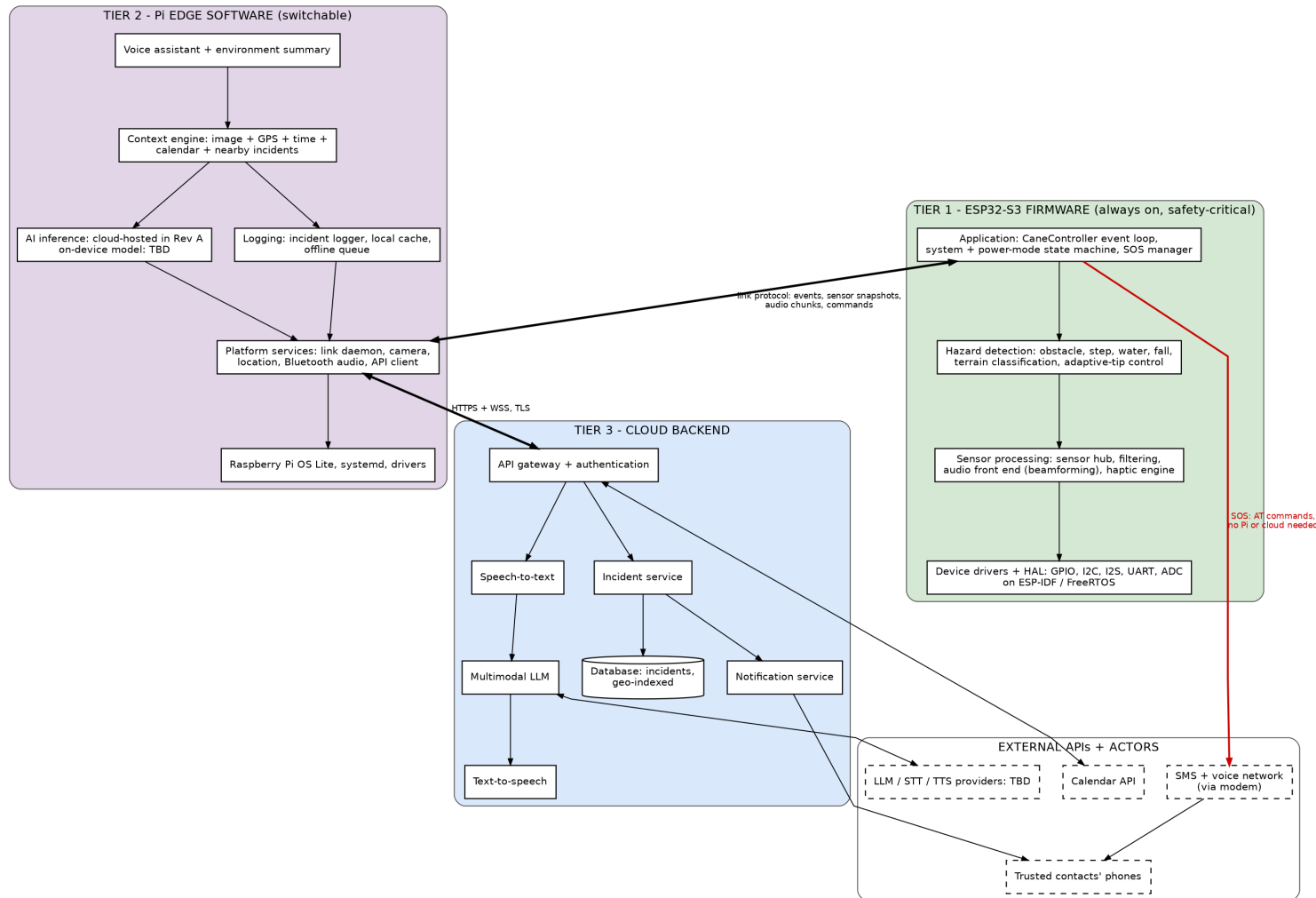
DIAGRAM 5 - HARDWARE / SOFTWARE INTERFACE (Rev A, proposed)



3.2 Diagram 6: software architecture

Software is three tiers: always-on ESP32-S3 firmware, switchable Pi edge software, and the cloud backend. The red arrow is the SOS path, which needs neither the Pi nor the cloud.

DIAGRAM 6 - SOFTWARE ARCHITECTURE (Rev A, proposed)



3.3 Software components

Hazard detection, haptics and SOS live entirely on the ESP32-S3, so they keep working when the Pi or the network is down.

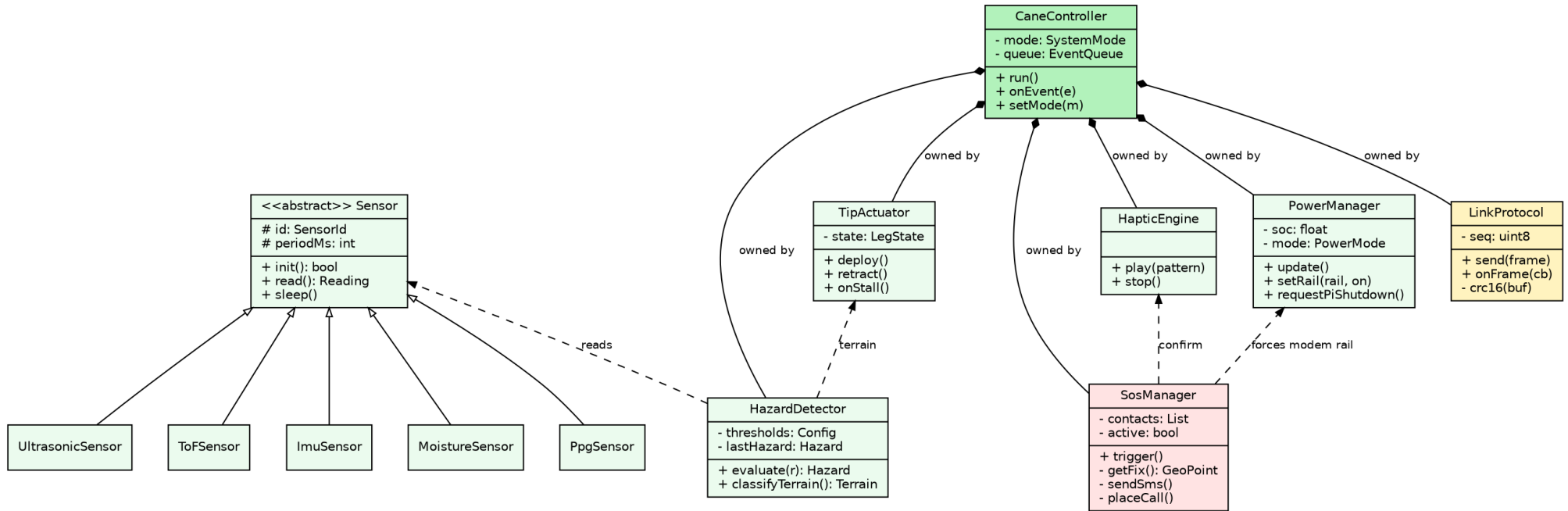
Node	Component	Responsibility
ESP32-S3	SensorHub	Schedules and reads ultrasonic, ToF, IMU, moisture and PPG
ESP32-S3	HazardDetector	Turns readings into obstacle, step, water, fall and terrain events
ESP32-S3	HapticEngine	Plays vibration patterns through the DRV2605L
ESP32-S3	TipActuator	Deploys and retracts the support legs, handles motor stall
ESP32-S3	AudioFrontEnd	Mic capture, beamforming and noise suppression; local tones
ESP32-S3	ButtonManager	Debounce, hold detection, wake from sleep
ESP32-S3	SosManager	SOS sequence: fix, SMS, call, beacon, all-clear
ESP32-S3	ModemDriver	AT commands to the SIM7600G-H
ESP32-S3	PowerManager	Battery state, rail switching, Pi power and shutdown handshake
ESP32-S3	LinkProtocol	Framed UART link to the Pi (COBS + CRC16)
Pi	LinkDaemon	Pi end of the link; publishes events to the other services
Pi	CameraService	On-demand frame capture
Pi	LocationService	GNSS position from the modem
Pi	ContextEngine	Combines image, GPS, time, calendar and nearby incidents
Pi	VoiceAssistant	Voice query round trip and spoken replies
Pi	EnvironmentSummary	Scene description on the E button
Pi	IncidentLogger	Stores image + GPS + time on the I button; nearby lookups
Pi	BluetoothAudio	A2DP stream to the bone-conduction headset
Pi	CloudClient	Single outbound HTTPS and WSS client with retry and offline queue
Cloud	API gateway + auth, SpeechToText, Multimodal LLM, TextToSpeech, Incident DB (geo-indexed), NotificationService, CalendarConnector	Heavy AI processing, storage and notifications

3.4 Interfaces

Interface	Transport	Messages or endpoints
ESP32-S3 to Pi	UART frames, COBS + CRC16	EVT_BUTTON, EVT_HAZARD, SENSOR_SNAPSHOT, AUDIO_CHUNK, PWR_STATE, CMD_HAPTIC, CMD_LOCAL_TONE, CMD_SHUTDOWN
Pi to cloud	HTTPS + WSS over TLS	/voice/query, /vision/describe, /incidents, /incidents/nearby, /sos/notify
ESP32-S3 to modem	AT commands over UART	SMS, voice call, GNSS fix; works with the Pi powered off

3.5 Class model (firmware)

CaneController is the firmware's root object: it owns the detector, actuators, power and SOS managers and the link, and drives them from one event queue.

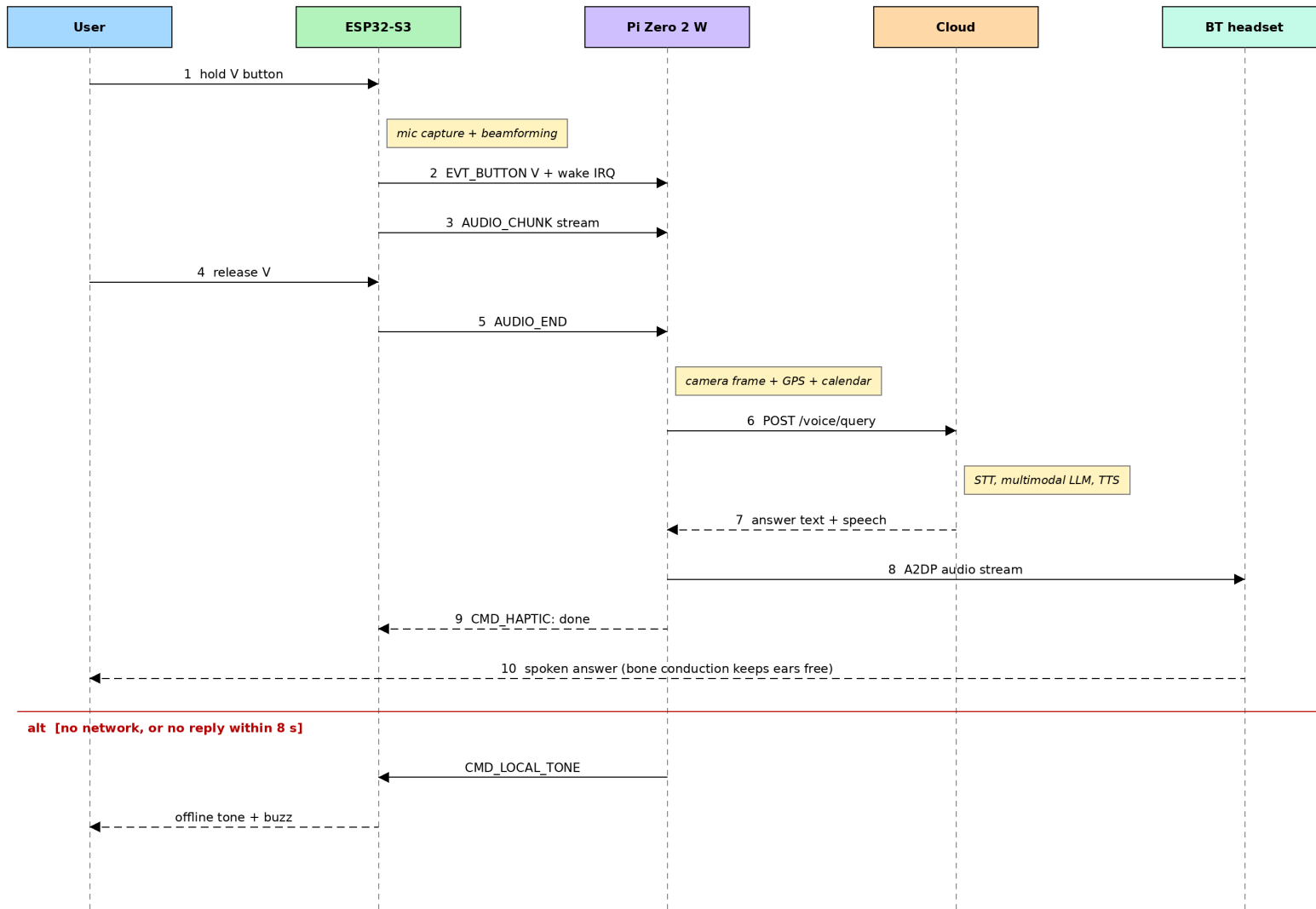


Pi class	Key members
VoiceAssistant	session; handleQuery(audio), speak(text)
ContextEngine	gps, time, calendar, recentIncidents; build() returns Context, describeScene(img)
IncidentLogger	local cache; log(img, gps, note), nearby(gps)
CloudClient	post(path, body), offline queue with retry

Enum	Values
SystemMode	BOOT, NORMAL, POWER_SAVE, SOS, CHARGING, FAULT
Hazard	OBSTACLE, STEP_UP, STEP_DOWN, WATER, FALL
LegState	STOWED, DEPLOYING, DEPLOYED, RETRACTING, STALLED

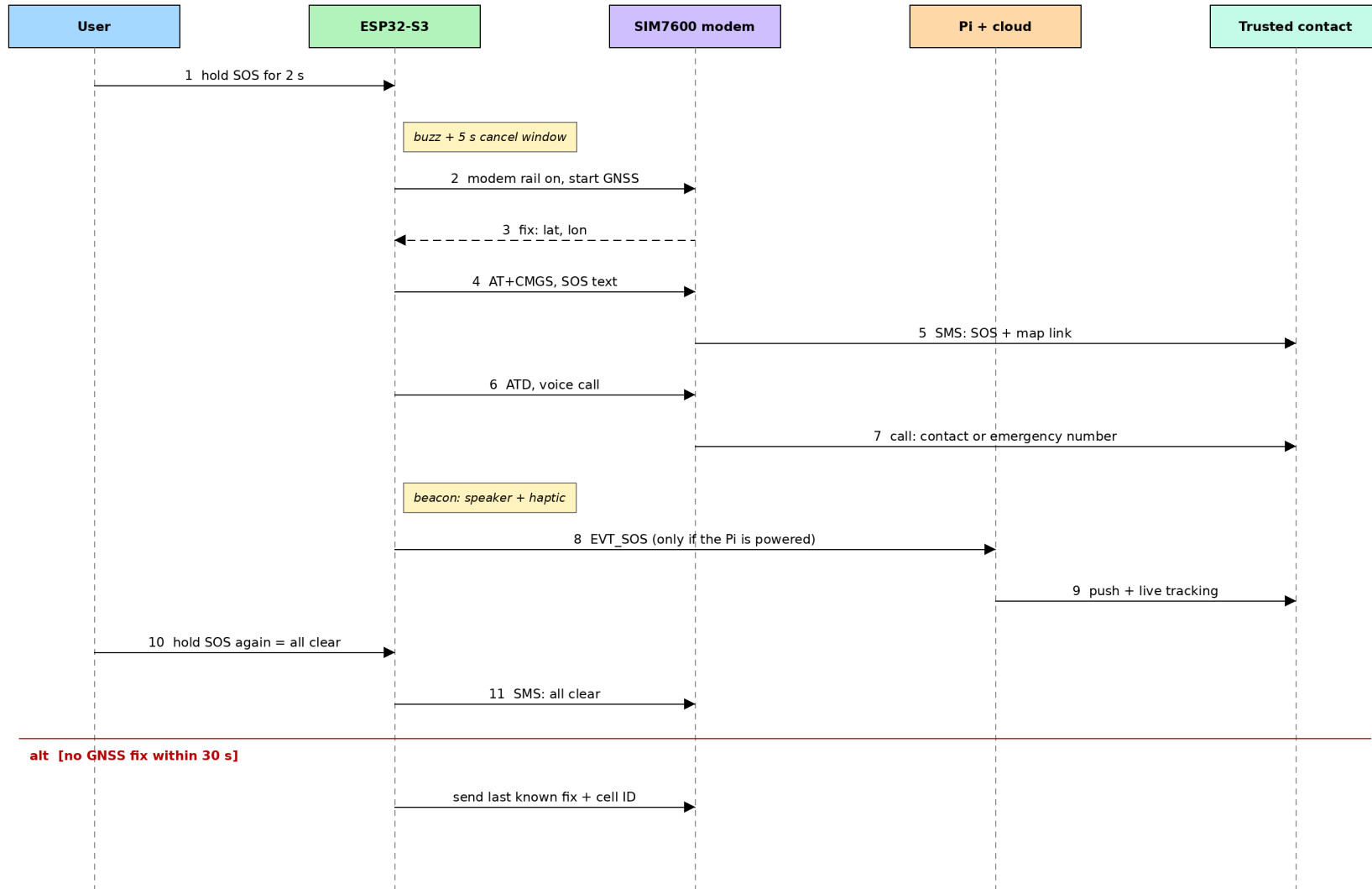
3.6 Sequence: Voice AI query

Audio is recorded only while the V button is held, and the answer comes back through the Bluetooth headset so the user's ears stay free.



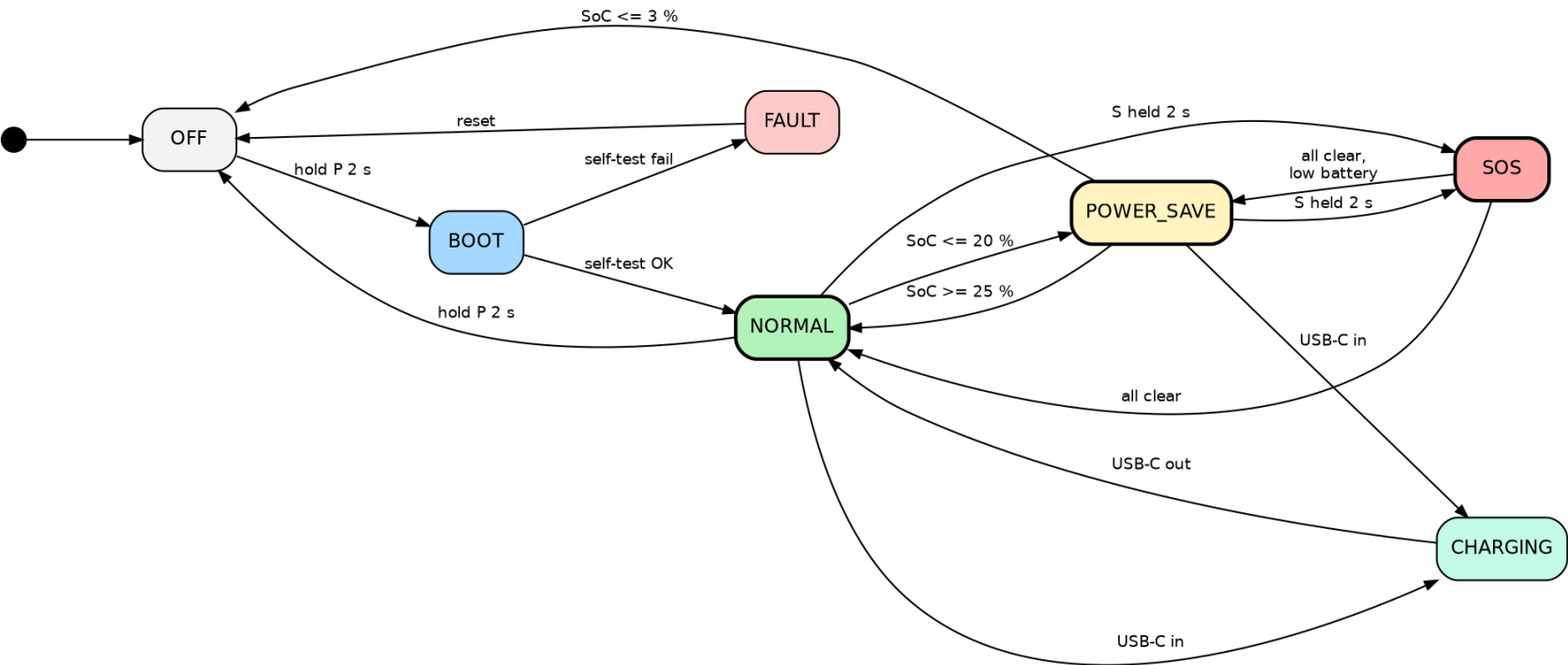
3.7 Sequence: SOS emergency

Steps 1 to 7 run on the ESP32-S3 and the modem alone, so SOS works in power-save mode, with the Pi off and with no phone nearby. The 2 s hold, 5 s cancel window, 8 s timeout and 30 s fix limit are starting values to tune with users.



3.8 State machine: system and power modes

The cane drops to POWER_SAVE at 20 % battery and returns to NORMAL at 25 %; the 5 % gap stops it flapping between modes. SOS is reachable from any running mode.

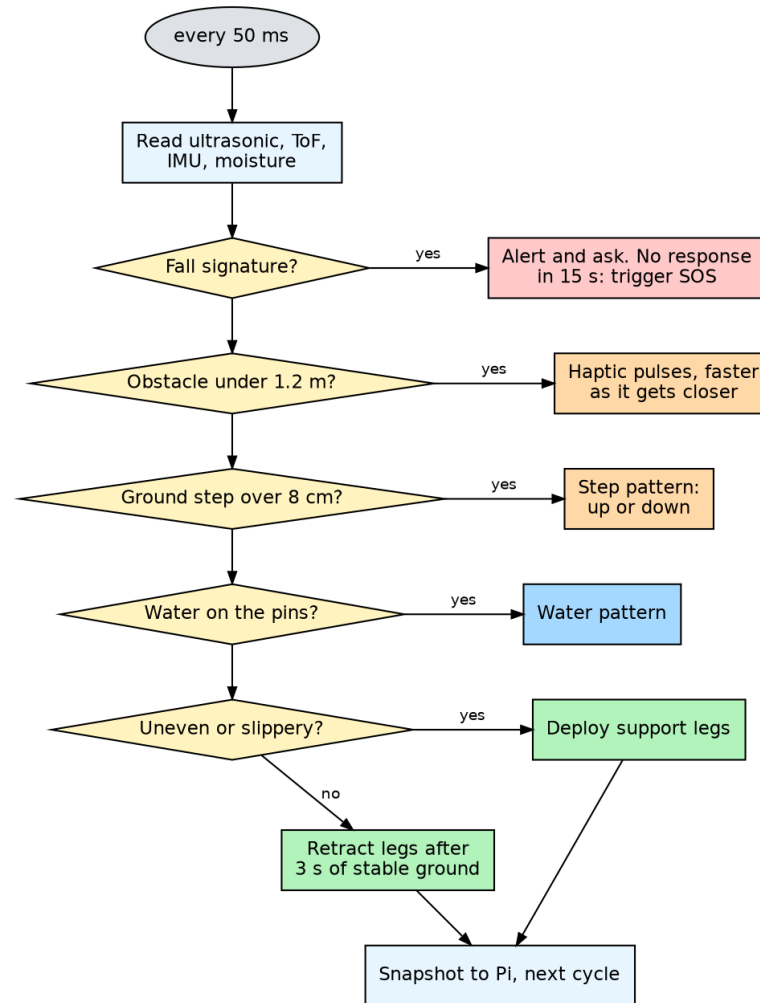


Mode	What runs
NORMAL	Camera, voice AI, cloud, all sensors, haptics; modem on standby
POWER_SAVE	Pi off. Ultrasonic, ToF, IMU and water sensing, haptics, adaptive tip and SOS only
SOS	Modem forced on, SMS and call, beacon, location update every 60 s
CHARGING	Sensors idle; returns to the mode it came from when unplugged
OFF	Entered after a clean Pi shutdown, or at 3 % to protect the cells after a final low-battery alert

The 20 % threshold follows the documentation's 15 to 20 % figure; final values need real battery data.

3.9 Safety loop (ESP32-S3, every 50 ms)

All five checks run every cycle, and the last one is what makes the support legs deploy automatically when the surface changes. If several checks fire in one cycle the alert priority is fall, then step, then obstacle, then water. The 1.2 m, 8 cm, 15 s and 3 s figures are example thresholds to tune in trials.

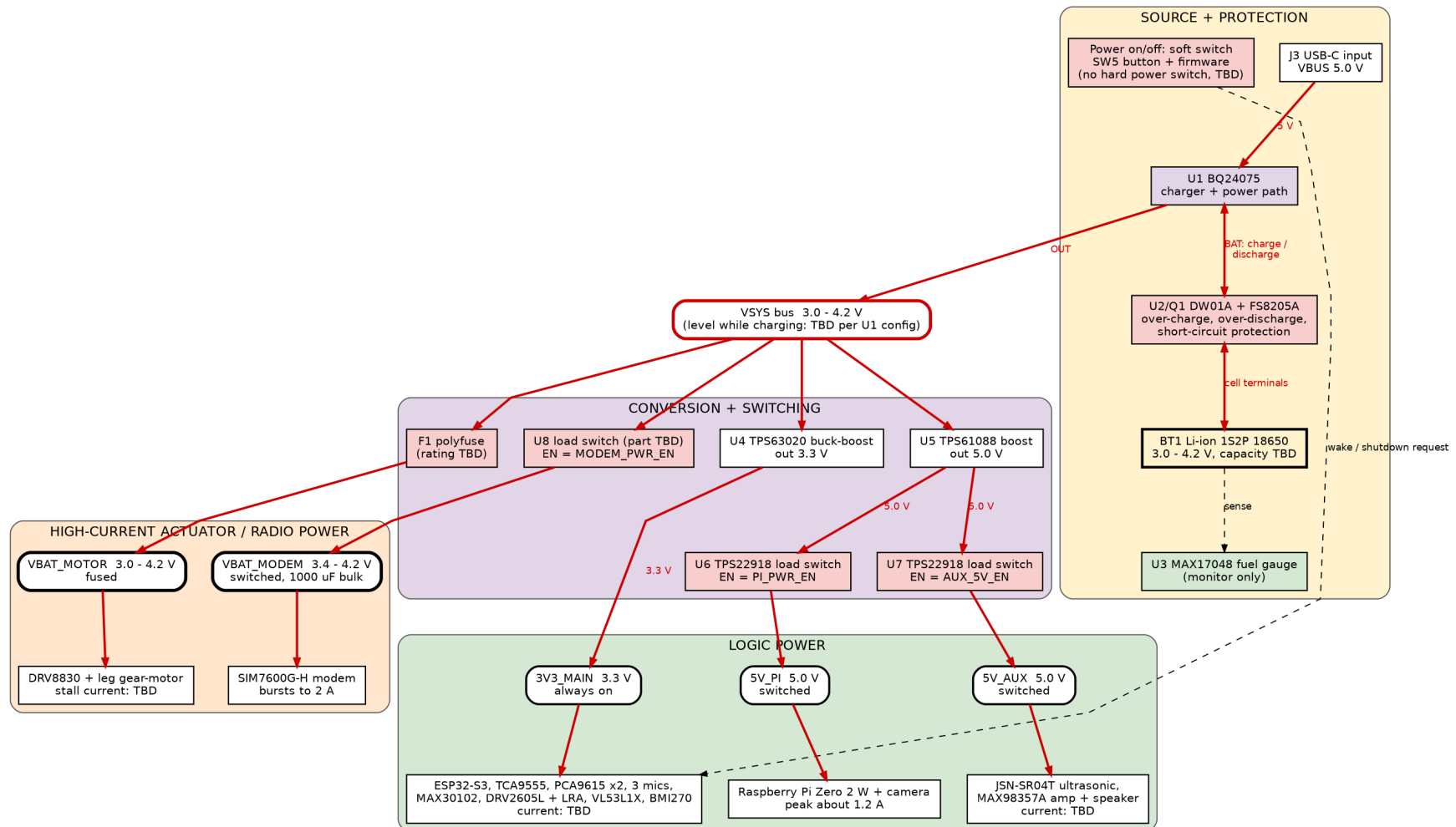


4. Part 3: power and wiring

4.1 Diagram 4: power distribution architecture

One single-cell pack (two 18650 cells in parallel, 3.0 to 4.2 V) feeds five rails; only 3V3_MAIN is always on, and the ESP32-S3 switches the rest through the TCA9555. Logic power (green) is kept separate from high-current modem and motor power (orange).

DIAGRAM 4 - POWER DISTRIBUTION ARCHITECTURE (Rev A, proposed)



4.2 Power rails

Rail	Source	Loads	Switched by	Notes
3V3_MAIN	TPS63020 buck-boost	ESP32-S3, TCA9555, mics, PPG, haptic driver, I2C buffer, ToF, IMU, NFC	Always on	Buck-boost because the cell ranges above and below 3.3 V
5V_PI	TPS61088 boost + TPS22918	Pi Zero 2 W, camera	TCA9555 P00	Peak about 1.2 A; off in POWER_SAVE
5V_AUX	TPS61088 boost + TPS22918	JSN-SR04T ultrasonic, MAX98357A amp	TCA9555 P03	Stays on in POWER_SAVE for the ultrasonic
VBAT_MODEM	Load switch from VSYS	SIM7600G-H	TCA9555 P06	3.4 to 4.2 V, 2 A bursts, 1000 uF bulk capacitor; forced on in SOS
VBAT_MOTOR	Polyfuse from VSYS, through J2	DRV8830 and leg gear-motor in the tip	DRV8830 over I2C	Current-limited for stall protection

The single-cell choice is deliberate: the modem can run straight from the battery and a one-cell charger is simple. All part numbers are proposed choices to confirm at schematic stage, and battery capacity is still to be decided.

4.3 Wiring harness

Two keyed connectors let the cane come apart for service: J1 (20 pins) between handle and shaft, and J2 (14 pins) between shaft and tip. The camera is not on J1; it has its own 22-pin CSI-2 FFC from the Pi, kept under 200 mm.

J1 pin	Signal (handle to shaft, 20-pin)	Connection	J2 pin	Signal (shaft to tip, 14-pin)	Connection
1	3V3_MAIN	Logic supply to the handle PCB	1	VBAT_MOTOR	Fused supply to the DRV8830
2	5V_AUX	Speaker amp supply	2	3V3_MAIN	ToF, IMU and buffer logic
3	GND	Power return (mates first)	3	5V_AUX	Ultrasonic supply
4	GND	Signal return	4	GND	Motor return (mates first)
5	I2C_SDA	ESP32 GPIO8 to PPG and haptic driver	5	GND	Signal return
6	I2C_SCL	ESP32 GPIO9 to PPG and haptic driver	6	DSCL+	Differential I2C clock, twisted with pin 7
7	BTN_V	To ESP32 GPIO6, active low	7	DSCL-	Pair with pin 6
8	BTN_E	To ESP32 GPIO7	8	DSDA+	Differential I2C data, twisted with pin 9
9	BTN_I	To ESP32 GPIO14	9	DSDA-	Pair with pin 8
10	BTN_S (SOS)	To ESP32 GPIO15, direct wire	10	US_TRIG	ESP32 GPIO4 to ultrasonic
11	BTN_P	To ESP32 GPIO16	11	US_ECHO	5 V pulse, through divider, to ESP32 GPIO5
12	I2S0_BCLK	ESP32 GPIO40 to mics 1 + 2	12	MOIST_DRV	ESP32 GPIO2 to excite pin
13	I2S0_WS	ESP32 GPIO41 to mics 1 + 2	13	MOIST_ADC	Sense pin to ESP32 GPIO1
14	I2S0_DIN	Mics 1 + 2 to ESP32 GPIO42	14	TIP_INT	ToF and IMU interrupt to TCA9555 P11
15	I2S1_BCLK	ESP32 GPIO38 to mic 3 and amp			
16	I2S1_WS	ESP32 GPIO39 to mic 3 and amp			
17	I2S1_DIN	Mic 3 to ESP32 GPIO21			
18	I2S1_DOUT	ESP32 GPIO47 to MAX98357A			
19	AMP_SD	TCA9555 P04 to amp shutdown			
20	SPARE	Reserved, or shield drain			

Harness rules

- Power and ground in 24 AWG; signals in 28 AWG.
- DSCL and DSDA run as two twisted pairs.
- The US_ECHO divider sits on the shaft PCB end.
- Spring-loaded pogo pins, with the ground pins mating first.
- TVS arrays on every J1 and J2 signal pin for ESD.
- Strain relief at both joints, sealed against water.
- Motor wires routed away from the I2S lines.

5. Design decisions and open points

Six decisions in this design are not in the project documentation and each one changes the wiring, so the team should review them before the schematic starts.

Decision	Reason
Headset audio comes from the Pi, not the ESP32-S3	The ESP32-S3 supports Bluetooth Low Energy only, which cannot stream audio to a headset. The ESP-to-Pi link uses the Pi's mini-UART so the Pi's Bluetooth stays free
All three mics connect to the ESP32-S3	The Pi has one audio input bus (two mics at most); the ESP32-S3 has two I2S peripherals
The modem has two connections: UART to the ESP32-S3, USB to the Pi	SOS must work with the Pi powered off, which POWER_SAVE requires
Single-cell battery (two cells in parallel)	The modem runs straight from the cell and charging stays simple
Tip sensors sit behind a differential I2C buffer	Plain I2C is unreliable over a cable of about 1.1 m
A TCA9555 expander carries slow control signals	The ESP32-S3 module runs out of safe pins

To verify before building

- Every part number, pin function and I2C address against its datasheet; this document was written from memory.
- Pi mini-UART RTS/CTS on GPIO16 and 17, and its reliability at the chosen baud rate.
- ESP32-S3 GPIO3 as the expander interrupt input (it is a strapping pin).
- Boot-log output on GPIO43 not disturbing the modem.
- SIM7600G-H UART and PWRKEY logic level against 3.3 V: a level translator is needed unless the breakout board includes one.
- MAX30102 needs a 1.8 V core supply: confirm the chosen module has its own regulator.
- Regulator, charger and load-switch choices, and the battery capacity.
- Thresholds in the state machine and safety loop, tuned in user trials.
- Which sensors trigger leg deployment; the documentation does not say, and the leg mechanism itself is still a concept.
- IMU location: the documentation places it in the shaft in one section and the tip in another; this design uses the tip.
- UWB, NFC and RFID are still options under evaluation; the Pi SPI pins are only reserved for UWB.
- Braille: the project documentation lists the E button with the L pattern (dots 1-2-3); the correct E is dots 1-5.